

# Reading and organizing data

Data Science Flipped Classroom - Module 4

Adriana Clavijo Daza

2026-05-07

# Reading data into R

# Why a dedicated package?

Base R already has `read.csv()` and `read.table()`, but the **tidyverse** reading packages have a few advantages:

- Faster on large files
- Return a `tibble` instead of a `data.frame`
- Preserve column names exactly as written, no `make.names()` mangling
- Consistent argument names across functions
- Clear messages about column types and parsing problems

# Two packages for two formats

## readr

- For **plain text** formats: CSV, TSV, and other delimited files.

```
1 library(readr)
```

## readxl

- For **Excel** files: `.xls` and `.xlsx`.

```
1 library(readxl)
```

Both are part of the tidyverse. `readr` loads with `library(tidyverse)`, but `readxl` must be loaded separately.

# The readr package

# The main reading functions

| Function                  | Reads files separated by            |
|---------------------------|-------------------------------------|
| <code>read_csv()</code>   | comma ,                             |
| <code>read_csv2()</code>  | semicolon ; (comma as decimal mark) |
| <code>read_tsv()</code>   | tab                                 |
| <code>read_delim()</code> | any delimiter you specify           |
| <code>read_fwf()</code>   | fixed width fields                  |
| <code>read_table()</code> | one or more spaces                  |

`read_csv2()` is what you usually need for files exported from European versions of Excel.

# A first read

```
1 penguins <- read_csv("data/penguins.csv")
```

- `readr` prints a short summary of the columns it parsed and their types.
- It is the easiest way to catch a column that came in as text when you expected a number.

# Key arguments of `read_csv()`

| Argument                | What it controls                                                             |
|-------------------------|------------------------------------------------------------------------------|
| <code>file</code>       | path or URL to the file                                                      |
| <code>col_names</code>  | column names: <code>TRUE</code> , <code>FALSE</code> , or a character vector |
| <code>col_types</code>  | column types: <code>NULL</code> (guess) or a spec                            |
| <code>col_select</code> | columns to keep, with <code>tidyselect</code>                                |
| <code>na</code>         | strings to treat as missing                                                  |
| <code>skip</code>       | rows to skip at the top                                                      |
| <code>n_max</code>      | maximum rows to read                                                         |
| <code>comment</code>    | character that starts a comment line                                         |
| <code>locale</code>     | decimal mark, encoding, time zone                                            |

# Specifying column types

You can let `readr` guess, or you can pin them down with a short spec.

```
1 penguins <- read_csv(  
2   "data/penguins.csv",  
3   col_types = "ccddddci"  
4 )
```

The letters mean: `c` character, `d` double, `i` integer, `l` logical, `D` date, `T` datetime, `f` factor, `?` guess, `-` skip.

# Specifying column types, the verbose way

For more control, use `cols()`:

```
1 penguins <- read_csv(  
2   "data/penguins.csv",  
3   col_types = cols(  
4     species = col_factor(),  
5     island = col_factor(),  
6     bill_length_mm = col_double(),  
7     year = col_integer()  
8   )  
9 )
```

# Reading only some columns

```
1 read_csv(  
2   "data/penguins.csv",  
3   col_select = c(species, island, starts_with("bill_"))  
4 )
```

`col_select` uses the same helpers as `dplyr::select()`: `starts_with()`, `ends_with()`, `contains()`, ranges with `:`.

# Locale

European data may use , as the decimal separator and . as the thousands separator.

```
1 read_csv2(  
2     "data/sales-eu.csv",  
3     locale = locale(decimal_mark = ",", grouping_mark = ".")  
4 )
```

# Writing data with `write_*`()

For every `read_*`() function, there is a **write counterpart**:

| Reading                   | Writing                    |
|---------------------------|----------------------------|
| <code>read_csv()</code>   | <code>write_csv()</code>   |
| <code>read_csv2()</code>  | <code>write_csv2()</code>  |
| <code>read_tsv()</code>   | <code>write_tsv()</code>   |
| <code>read_delim()</code> | <code>write_delim()</code> |

# Example

```
1 write_csv(penguins, "data/penguins-clean.csv")
```

`write_csv()` does not write row names and uses "" for missing values by default. Both choices are deliberate and play well with re-reading the file later.

# The readxl package

# Reading Excel files

`readxl` handles both `.xls` and `.xlsx` without external dependencies.

| Function                  | What it reads              |
|---------------------------|----------------------------|
| <code>read_excel()</code> | guesses from the extension |
| <code>read_xls()</code>   | only <code>.xls</code>     |
| <code>read_xlsx()</code>  | only <code>.xlsx</code>    |

# A first Excel read

```
1 sales <- read_excel("data/sales.xlsx")
```

By default, `read_excel()` reads the first sheet, treats the first row as column names, and guesses types from the first 1000 rows.

# Key arguments of `read_excel()`

| Argument               | What it controls                                               |
|------------------------|----------------------------------------------------------------|
| <code>path</code>      | path to the workbook                                           |
| <code>sheet</code>     | sheet name or number                                           |
| <code>range</code>     | a cell range like <code>"B3:D87"</code>                        |
| <code>col_names</code> | <code>TRUE</code> , <code>FALSE</code> , or a character vector |
| <code>col_types</code> | type per column                                                |
| <code>na</code>        | strings to treat as missing                                    |
| <code>skip</code>      | rows to skip at the top                                        |
| <code>n_max</code>     | maximum rows to read                                           |
| <code>trim_ws</code>   | trim white space around values                                 |

# Picking a sheet

```
1 read_excel("data/sales.xlsx", sheet = "Q1")  
2 read_excel("data/sales.xlsx", sheet = 2)
```

# Picking a cell range

```
1 read_excel(  
2     "data/sales.xlsx",  
3     sheet = "Summary",  
4     range = "B3:F50"  
5 )
```

`range` is useful when the spreadsheet has titles, blank rows, or notes above the data table. Pick the rectangle of actual data.

# Specifying column types

`col_types` accepts one value per column from this set:

"skip", "guess", "logical", "numeric", "date", "text", "list".

```
1 read_excel(  
2   "data/sales.xlsx",  
3   col_types = c("text", "date", "numeric", "numeric")  
4 )
```

# Writing Excel files

`readxl` is read-only. To write an Excel file, use the companion package `writexl`:

```
1 library(writexl)
2
3 write_xlsx(sales, "data/sales-clean.xlsx")
```

`writexl::write_xlsx()` can write a list of data frames to multiple sheets in one call: `write_xlsx(list(Q1 = q1, Q2 = q2), "sales.xlsx")`.

# Categorical variables: factors

# What is a factor?

A **factor** is R's data type for categorical variables.

Internally, R stores the values as integer codes plus a vector of **levels** that gives each code a label.

```
1 cuts <- factor(c("Good", "Ideal", "Fair", "Ideal", "Good"))  
2 cuts
```

```
[1] Good Ideal Fair Ideal Good  
Levels: Fair Good Ideal
```

# Why use factors?

- They make the set of allowed values explicit
- They control the order of categories in tables, plots, and models
- They help statistical functions know that a variable is categorical
- They are smaller in memory than repeated character strings

# The factor() function

```
1 factor(  
2   x,                # the data, usually a character vector  
3   levels = ...,     # the allowed values, in the order you want  
4   labels = levels,  # optional pretty names for the levels  
5   ordered = FALSE   # is there a natural ranking?  
6 )
```

# Setting levels explicitly

Without `levels`, R picks them in alphabetical order, which is rarely what you want.

```
1 factor(c("low", "high", "medium")) |> levels()
```

```
[1] "high"    "low"     "medium"
```

With `levels`, you set the order yourself.

```
1 factor(  
2   c("low", "high", "medium"),  
3   levels = c("low", "medium", "high")  
4 ) |> levels()
```

```
[1] "low"     "medium"  "high"
```

# Levels also act as a check

Values that are not in `levels` become `NA`.

```
1 factor(  
2   c("low", "med", "high"),  
3   levels = c("low", "medium", "high")  
4 )
```

```
[1] low  <NA> high  
Levels: low medium high
```

This is a useful guard against typos in survey data or data entry.

# Pretty labels

`labels` lets you replace the raw values with clearer names without changing the underlying data.

```
1 factor(  
2   c(1, 2, 1, 3, 2),  
3   levels = c(1, 2, 3),  
4   labels = c("Low", "Medium", "High")  
5 )
```

```
[1] Low      Medium Low      High      Medium  
Levels: Low Medium High
```

# Ordered factors

An **ordered factor** adds a natural ranking to the levels.

```
1 risk <- factor(  
2   c("low", "high", "medium", "low", "high"),  
3   levels = c("low", "medium", "high"),  
4   ordered = TRUE  
5 )  
6 risk
```

```
[1] low    high   medium low    high
```

```
Levels: low < medium < high
```

`ordered(x, levels)` is a shortcut for `factor(x, levels, ordered = TRUE)`.

# Factor vs ordered: comparisons

Ordered factors support `<`, `>`, `<=`, `>=`. Unordered factors do not.

```
1 cuts
```

```
[1] Good Ideal Fair Ideal Good  
Levels: Fair Good Ideal
```

```
1 risk
```

```
[1] low high medium low high  
Levels: low < medium < high
```

```
1 risk[1] < risk[2]
```

```
[1] TRUE
```

```
1 cuts[1] < cuts[2]
```

```
[1] NA
```

# Factor vs ordered: in models

When you put a factor in `lm()`, R uses contrasts to encode it.

## Unordered factor

Treatment contrasts: each non-reference level vs a baseline.  
Coefficients read as “level X minus reference.”

## Ordered factor

Polynomial contrasts: linear, quadratic, cubic, ... trends across the ordered levels.  
Coefficients read as overall shape, not pairwise.

# A note on forcats

The tidyverse package `forcats` provides helpers for working with factors:

- `fct_relevel()`: change the order of levels by hand
- `fct_infreq()`: order levels by frequency
- `fct_reorder()`: order levels by another variable
- `fct_recode()`: rename levels
- `fct_lump()`: collapse rare levels into “Other”

We will not cover them in detail today, but they are worth knowing about.

# Data organization in spreadsheets

# Why this matters

Most data still arrive as a spreadsheet someone built by hand.

How that spreadsheet is organized determines whether the data can be read, joined, analyzed, and trusted later, including by you, six months from now.

The next slides summarize a short, practical paper that has become a standard reference.

# Source

## Attribution

These principles are taken from:

Broman, K. W., & Woo, K. H. (2018). Data Organization in Spreadsheets. *The American Statistician*, 72(1), 2-10. <https://doi.org/10.1080/00031305.2017.1375989>

Companion tutorial: <https://kbroman.org/dataorg/>

All examples and recommendations in this section are theirs. Any errors in the summary are mine.

# The 12 principles

1. Be consistent
2. Choose good names for things
3. Write dates as YYYY-MM-DD
4. No empty cells
5. Put just one thing in a cell
6. Make it a rectangle
7. Create a data dictionary
8. No calculations in the raw data files
9. Do not use font color or highlighting as data
10. Make backups
11. Use data validation to avoid data entry errors
12. Save the data in plain text files

# 1. Be consistent

Use the same spelling, capitalization, and code throughout the file.

## Inconsistent

Male, male, M, m, 1

## Consistent

male, male, male, male, male

The same applies to missing values, units, ID formats, and date formats. Pick one convention per variable and never break it.

## 2. Choose good names for things

- No spaces: use `_` or `-`
- No special characters: `/`, `?`, `%`, `$`, `*`, parentheses
- Be descriptive but short: `glucose_mg_dl` beats `glu` and `blood_glucose_concentration_milligrams_per_deciliter`
- Stick to ASCII: avoid accented letters in column names

### 3. Write dates as YYYY-MM-DD

The ISO 8601 format is unambiguous and sorts correctly as text.

Risky

03/04/2025

(March 4? April 3?)

Safe

2025-03-04

Excel actively rewrites dates. Store dates as text with the YYYY-MM-DD format, or as a true date column you control, and check before saving.

## 4. No empty cells

Empty cells are ambiguous: missing? not applicable? not measured yet?

- Use an explicit code for missing: **NA**, or whatever your team agrees on
- If a header spans many rows visually, repeat it on every row
- Never leave a column blank to mean “same as the row above”

## 5. Put just one thing in a cell

One thing per cell

| height_cm | weight_kg |
|-----------|-----------|
| 172       | 68        |

Two things squeezed in

| measurements   |
|----------------|
| 172 cm / 68 kg |

If a cell contains units, comments, or a range, those belong in their own column.

## 6. Make it a rectangle

The data should be one rectangle: one header row at the top, one observation per row, one variable per column, no blank rows, no merged cells.

Multi-row headers, side tables, totals at the bottom, and color-coded subsections all break the rectangle and break automated reading.

## 7. Create a data dictionary

A separate file (or sheet) that lists, for every variable:

- The variable name as it appears in the data
- A longer human description
- Units
- Allowed values or range
- The code used for missing

This is the document a new collaborator should be able to read in five minutes and understand the dataset.

## 8. No calculations in the raw data files

Keep the raw data file purely for data.

- No formulas, no totals, no derived columns
- Do calculations in a separate analysis script (R, Python, ...)
- This way, anyone can reproduce the analysis from the same raw file

## 9. Font color and highlighting are not data

If yellow rows mean “outlier” and red cells mean “needs review”, that information will be lost the moment the file is exported to CSV.

Encode the meaning in a real column: `outlier = TRUE`, `status = "needs_review"`. Then color the cells too if it helps you read the spreadsheet.

## 10. Make backups

- Keep multiple copies, in multiple places
- Use version control (git) for analysis code, and a versioned drive (or git LFS, or a backup service) for data files
- The raw data file should be **read only** once collected

# 11. Use data validation

Most spreadsheet programs (Excel, LibreOffice, Google Sheets) let you restrict what can go in a cell:

- A drop-down list of allowed categories
- A numeric range
- A required date format

This catches typos at entry time, when they are cheap to fix.

## 12. Save the data in plain text

Plain text formats (`.csv`, `.tsv`) are:

- Readable by every tool, on every system, today and in 30 years
- Independent of any vendor's file format changes
- Easy to put under version control and to diff

Excel `.xlsx` is fine as a working format. The archived version of the dataset should be a plain text file with a clearly named encoding.

# A short checklist before sharing data

- One rectangle, one header row, one observation per row
- Consistent codes, consistent missing-value marker, consistent date format
- Names without spaces or special characters
- A data dictionary alongside the file
- No formulas, no formatting that carries meaning

# Learn more

<https://kbroman.org/dataorg/>

## Reference

Broman, K. W., & Woo, K. H. (2018). Data Organization in Spreadsheets. *The American Statistician*, 72(1), 2-10.

# Questions?